

Ingeniería de Software I

Avance de Proyecto #5 — Sprint 2: interfaz funcional y aplicación de patrones de diseño

Fechas del Sprint: Semana 10 – Semana 12.

Docente: Juan Pablo Bustamante Moreno.

Escuela Tecnológica Instituto Técnico Central (ETITC).

En el Sprint 1, cada equipo construyó un primer incremento del proyecto: una API funcional conectada a una base de datos, ejecutable mediante Docker y trabajada colaborativamente mediante GitHub. También documentó su organización mediante un diagrama de paquetes y registró la planificación, seguimiento y cierre del Sprint.

Durante el Sprint 2 deberán ampliar ese incremento con una interfaz mínima que permita validar las funcionalidades implementadas y mejorar la organización del código mediante patrones de diseño.

La pregunta central de este avance es:

¿El equipo logró integrar una interfaz funcional y aplicar patrones de diseño pertinentes, manteniendo coherencia entre las historias de usuario, la arquitectura y el código?

1. Incremento funcional

El equipo debe continuar desarrollando las historias priorizadas para el Sprint 2 y conservar el funcionamiento alcanzado en el Sprint 1.

El incremento debe incluir:

- API funcional y conectada a la base de datos del proyecto.
- Ejecución reproducible de la API y la base de datos mediante Docker Compose.
- Persistencia de datos y volumen de Docker.
- Variables de entorno y ausencia de credenciales sensibles en el repositorio.

- Evidencia de colaboración mediante GitHub: commits, ramas, Pull Requests y revisión de cambios.
- Manejo básico de errores y validaciones.
- Una o dos pantallas funcionales conectadas con la API.
- Implementación de DAO y de los patrones de diseño solicitados en el siguiente apartado.

En cuanto a la interfaz, implementen una o dos pantallas que permitan a un usuario ejecutar y validar funcionalidades reales del proyecto. Todavía no se espera una interfaz completa ni un diseño visual sofisticado.

Las pantallas deben:

- Corresponder a las historias seleccionadas para estos Sprints.
- Consultar y/o modificar datos mediante la API, según el flujo elegido.
- Mostrar información real obtenida desde la base de datos.
- Informar el resultado de las operaciones: confirmación, validación o error.
- Permitir recorrer al menos un flujo funcional completo.

Por ejemplo, en un sistema de reservas:

- Una pantalla para consultar reservas.
- Una pantalla o formulario para crear o actualizar una reserva.
- Mensajes para confirmar la operación o explicar por qué no pudo realizarse.

No se consideran suficientes pantallas estáticas, mockups, formularios sin conexión o interfaces que utilicen exclusivamente datos simulados. Si logran llegar más lejos de lo que se solicita, ¡mejor!

Incluyan, en su documento, capturas de las pantallas y una explicación breve del flujo que permiten validar, señalando las historias de usuario relacionadas.

2. Patrones de diseño

El equipo debe implementar el patrón DAO de manera obligatoria y DTO cuando sea pertinente. Además, debe implementar al menos dos patrones creacionales, dos estructurales y dos de comportamiento.

DAO — Obligatorio

El acceso a datos debe estar organizado mediante DAO —Data Access Object—, separando las operaciones de persistencia de los controladores, la interfaz y las reglas de negocio.

La implementación debe evidenciar:

- Uno o más DAO asociados con las entidades o recursos trabajados.
- Uso del DAO desde los servicios o componentes correspondientes.
- Ausencia de consultas o lógica de acceso a datos dispersas en la interfaz y los controladores.

Usar un ORM no reemplaza por sí solo esta exigencia: el equipo debe mostrar cómo encapsula el acceso a datos mediante DAO, aunque internamente utilice el ORM.

DTO — Si aplica

Implementen DTO —Data Transfer Object— cuando necesiten definir estructuras específicas para recibir o enviar información entre la interfaz, la API y otros componentes. Si lo implementan, expliquen:

- Qué información transporta.
- En qué flujo se utiliza.
- Por qué esa estructura se diferencia de la entidad persistida.

Si consideran que no es necesario en este incremento, incluyan una justificación breve en su documento.

Patrones creacionales, estructurales y de comportamiento

Cada equipo debe seleccionar patrones que respondan a problemas concretos de su proyecto. Para cada patrón seleccionado:

- Debe existir una implementación identificable en el código.
- Debe resolver un problema concreto de creación, estructura o comportamiento.
- El equipo debe explicar, en el documento, qué beneficio aporta y qué complejidad introduce.

Categoría	Cantidad	Ejemplos de opciones
Creacionales	al menos 2 patrones distintos	Factory Method, Abstract Factory, Builder, Prototype, Singleton

Estructurales	al menos 2 patrones distintos	Adapter, Facade, Decorator, Composite, Proxy, Bridge
De comportamiento	al menos 1 patrón	Strategy, Observer, State, Command, Template Method, Chain of Responsibility

Si quieren implementar más patrones de los que se solicitan, ¡adelante!

3. Registro documental del Sprint

Mantengan la misma estructura de registro utilizada en el Sprint 1, ajustando las referencias al Sprint 2.

a) Registro del Sprint Planning.

Elemento	Descripción requerida
Sprint Goal	Objetivo concreto, demostrable y orientado a valor
Historias seleccionadas	Historias de usuario priorizadas para el sprint, con referencia a los criterios que deben cumplirse
Tareas técnicas	Tareas necesarias para construir el incremento
Responsables iniciales	Responsable inicial por tarea; no implica que el trabajo no pueda ser colaborativo
Dependencias o riesgos	Riesgos, bloqueos o incertidumbres identificadas
Definition of Done	Condiciones que usará el equipo para considerar una historia terminada

Además de la tabla, incluyan una captura del tablero con las historias seleccionadas para Sprint 2.

b) Registro de un Daily Scrum.

El equipo debe documentar al menos un seguimiento diario o reunión breve de sincronización durante el Sprint. La evidencia puede ser:

- Captura de una reunión virtual.
- Foto de una reunión presencial.
- Registro de una videollamada.

También deben completar la siguiente tabla:

Integrante	¿Qué terminé desde el último seguimiento?	¿Qué haré a continuación?	¿Qué bloqueo o ayuda necesito?

Opcionalmente, podrían incluir una captura del tablero que evidencie el progreso tras la reunión del equipo.

c) Sprint Review y Sprint Retrospective

El equipo debe realizar una revisión del incremento funcional antes de cerrar el Sprint, de acuerdo con la validación que se realizará en clase el próximo viernes. La evidencia puede ser:

- Video breve de prueba.
- Capturas de Postman/Insomnia/Thunder Client.
- Captura de contenedores en ejecución.

Incluyan la siguiente tabla:

Historia	¿Se cumplió?	Criterios de aceptación pendientes, ajustes u observaciones
HU-01	Sí / Parcialmente / No	
HU-02	Sí / Parcialmente / No	

A partir de esta revisión, construyan la retrospectiva del Sprint. En una serie de párrafos, aborden los siguientes ítems:

- ¿Qué funcionó bien?
- ¿Qué no funcionó o dificultó el trabajo?

- ¿Qué aprendimos?
- ¿Qué acción concreta aplicaremos en el siguiente corte?

La última respuesta debe ser una acción concreta y verificable, no una frase general como “trabajaremos mejor” o “nos organizaremos más”.

Entrega

Actualicen el documento compartido de OneDrive creado para los avances de proyecto en el corte previo. No deben crear un archivo nuevo: el documento del proyecto va a crecer de manera progresiva durante todo el semestre.

Este avance no se califica de forma independiente, sino que hará parte de la evidencia evaluada en la sustentación del segundo corte. Agreguen al documento un enlace al github de su equipo (debe estar público), la explicación de los patrones y el registro documental del Sprint 2.

Para la sustentación, bastará con mostrar su proyecto en ejecución y el diagrama de paquetes solicitado en el avance previo. Se realizará el día **23 de octubre de 2026**.